



Licence MIASHS première année

Rapport de projet informatique

Captcha sous LLM

Projet réalisé du 7 octobre 2025 au 6 janvier 2026

Github : [PRIX000/CAPTCHA-IA: A Captcha using LLM](https://github.com/PRIX000/CAPTCHA-IA)
(<https://github.com/PRIX000/CAPTCHA-IA>)

Clé Gemini : CLE TRANSMISE SUR COURSENLIGNE
(pour des raisons de sécurité je ne vais pas transmettre ma clé sur un dépôt public)

Membres du groupe

KEBBACHE WALID - N°Etudiant : 45003317
YAHOU SARAH - N°Etudiant : 45011767

Remerciements

Merci, merci à tous.

Je remercie GEMINI et ChatGPT de m'avoir grandement aidé pour le développement, qui ont agi comme de véritables **interprètes de pensée** durant ce projet.

Si l'idée originale du Dashboard sous Tkinter, l'architecture globale de **VerifBot** ainsi que **Trainmodel** viennent entièrement de nous, l'IA a été notre bras droit pour la mise en œuvre technique.

Nous lui avons **soufflé** nos concepts, nos structures et nos exigences ergonomiques, et elle s'est chargée de traduire nos idées en lignes de code fonctionnelles.

Cette collaboration nous a permis de nous concentrer sur la logique métier et la conception du système, tout en déléguant la rédaction fastidieuse du code.

Ce rapport est donc le résultat d'un travail d'équipe où l'humain a gardé la vision créative et l'IA a assuré la main-d'œuvre numérique.

Table des matières

1	Introduction	4
2	Environnement de travail	4
3	Description du projet et objectifs	4
3.1	Fonctionnement de la collecte	5
3.2	Objectifs principaux	5
4	Bibliothèques, Outils et technologies	6
5	Travail réalisé	6
6	Difficultés rencontrées	7
7	Bilan	7
7.1	Conclusion	7
7.2	Perspectives	8
8	Bibliographie	8
9	Webographie	8
10	Annexes	9
A	Cahier des charges	9
B	Exemple d'exécution du projet	11
C	Manuel d'utilisation	14

1 Introduction

Dans un web de plus en plus automatisé, la distinction entre un utilisateur humain et un robot est devenue un enjeu majeur de cybersécurité. Ce projet, intitulé **Captcha sous LLM**, vise à concevoir une solution complète de CAPTCHA comportemental.

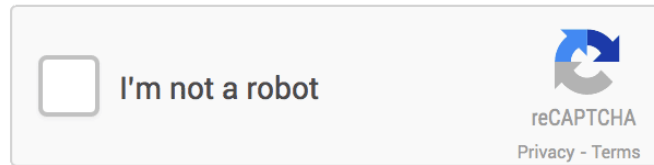


FIGURE 1 – Exemple de reCAPTCHA

Contrairement aux CAPTCHA traditionnels basés sur la lecture de texte ou la sélection d'images, notre système analyse le comportement de la souris (vitesse, accélération, hésitations, régularité des trajectoires) afin de déterminer la nature de l'utilisateur.

Le projet ne se limite pas à la détection : nous avons également développé un bot attaquant capable de **voir** l'écran et de simuler un comportement humain, afin de tester la robustesse de notre propre système.

Notre détecteur est un système hybride : un modèle de Machine Learning prend la décision principale, et en cas d'incertitude (**zone de doute**), un modèle de langage (LLM) est sollicité comme second avis afin d'améliorer la robustesse de la détection.

L'objectif final est de proposer une architecture client-serveur fonctionnelle, pilotée par un Dashboard d'administration centralisé, permettant de lancer l'ensemble sans manipulations complexes en ligne de commande.

2 Environnement de travail

Ce projet *Fullstack* a nécessité la maîtrise d'un écosystème technique hétérogène, du développement web jusqu'à l'IA et l'automatisation système.

Le développement a été pensé *cross-platform*, avec compatibilité Windows, macOS et Linux.

- Langages : Python 3.10+ (Backend, IA, scripts), TypeScript/React (Frontend), Batch/Bash (automatisation).
- Outils : Git pour le versionnage, Visual Studio Code pour le développement.
- Architecture cible : Client (navigateur) ↔ API REST (FastAPI) ↔ Modèles IA (locaux & cloud).

3 Description du projet et objectifs

Le projet s'articule autour de quatre piliers interconnectés, chacun jouant un rôle spécifique dans la chaîne d'acquisition–analyse–décision.

- **Frontend (l'épreuve)** : application web proposant trois mini-jeux (Trace, Puzzle, Sélection d'images) conçus pour récolter des données biométriques issues des mouvements de souris.
- **Backend & IA (le juge)** : API recevant les données, extrayant des caractéristiques cinématiques, puis renvoyant un verdict **Humain** ou **Robot** via un modèle de ML hybride.

- **Bot attaquant (l'adversaire)** : script autonome utilisant OpenCV pour repérer des éléments à l'écran et une IA générative (Gemini) pour simuler des trajectoires réalistes.
- **Dashboard Admin (l'orchestrateur)** : interface Tkinter permettant de lancer et gérer tous ces services sans passer par le terminal.

3.1 Fonctionnement de la collecte

Le navigateur collecte les points (x, y, t) durant l'interaction avec les jeux, puis les envoie au backend.

Le backend normalise et transforme ces données en *features* physiques (vitesse, accélération, rectitude...), et produit une prédiction via le modèle ML.



```
{
  "sessionId": "8c167781-ead8-47e2-bc19",
  "taskId": "image-select",
  "timestamp": "2025-12-26T14:15:43.974",
  "userAgent": "Mozilla/5.0 (Windows NT",
  "screen": {
    "width": 1536,
    "height": 864
  },
  "events": [
    {
      "x": 777,
      "y": 462,
      "t": 16554.5
    },
    {
      "x": 780,
      "y": 466,
      "t": 16560.2999999998882
    },
    {
      "x": 780
    }
  ]
}
```

FIGURE 2 – Exemple de données collectées (JSON)

En cas d'incertitude, une logique de **zone de doute** déclenche une analyse complémentaire par Gemini afin d'améliorer la robustesse de la décision finale.

3.2 Objectifs principaux

- Concevoir des épreuves courtes et exploitables (capables de capturer de la motricité fine).

- Construire une chaîne de traitement reproductible (normalisation, extraction de features, entraînement, évaluation).
- Démontrer la solidité du système en développant un bot attaquant réaliste.
- Proposer une expérience **clé en main** via un dashboard de lancement.

4 Bibliothèques, Outils et technologies

Nous avons exploité des bibliothèques spécialisées pour chaque brique du projet :

- Frontend : React, TypeScript, Vite.
- Backend & API : FastAPI + Uvicorn (serveur ASGI).
- Exposition : Ngrok (tunneling).
- Data science / ML : Scikit-learn (Random Forest), Pandas / NumPy, Joblib (sérialisation du modèle).
- Vision & IA générative : OpenCV (cv2), Google Generative AI (Gemini).
- Automatisation : PyAutoGUI, subprocess, threading.

5 Travail réalisé

A. Développement des épreuves CAPTCHA (Frontend)

Nous avons créé trois composants React distincts afin de capturer différents aspects de la motricité :

- **TraceGame** : suivi d'une ligne sinusoïdale, avec mesure de la déviation par rapport à une trajectoire attendue.
- **PuzzleGame** : *drag & drop* d'une pièce vers une zone cible, utile pour analyser accélération/décélération à l'approche.
- **ImageSelectGame** : sélection d'images (ex. chats/chiens/voitures) avec tolérance à l'erreur pour mimer le comportement humain.

Une page **Résultats** permet d'exporter les sessions (par épreuve et cumulées) et de lancer la vérification via le backend, puis d'afficher le verdict final.

Chaque mouvement de souris est enregistré sous forme de séries temporelles (X, Y, Temps) puis envoyé au serveur pour analyse.

B. Moteur de détection (Backend & ML)

Le backend implémente une chaîne de traitement complète :

- *Feature engineering* (`utilitis.py`) : transformation des données brutes en indicateurs physiques (vitesse moyenne, variabilité, accélération, *straightness*, etc.).
- Modèle Random Forest (`train.py`) : entraînement sur notre dataset Humains vs Bots, avec validation croisée (Stratified K-Fold).
- Architecture hybride (`api.py`) : logique de doute :
 - Si confiance forte, le Random Forest tranche.
 - Si confiance intermédiaire, appel à Gemini pour une analyse contextuelle des logs et de la cohérence des patterns.

C. Bot attaquant (simulation d'attaque)

Pour éprouver notre défense, nous avons développé un bot autonome :

- Vision : capture écran et détection d'éléments via filtrage HSV / traitement d'images (OpenCV) afin de repérer boutons et cibles.
- Trajectoires réalistes : génération d'une trajectoire (JSON) par Gemini, puis exécution avec PyAutoGUI en ajoutant des micro-pauses et imperfections.

D. Dashboard d'administration (Tkinter)

Le dashboard (`APP.py`) automatise l'installation et le lancement :

- Installation partielle des dépendances (librairies)
- Lancement parallèle (API + React) via multi-threading, garantissant une interface réactive.

6 Difficultés rencontrées

- Normalisation : les résolutions d'écran différentes imposaient de normaliser les coordonnées entre 0.0 et 1.0 pour rendre le modèle généralisable.
- Latence & asynchronisme : concilier FastAPI (async) et des appels Gemini parfois lents sans bloquer l'expérience.
- Vision & scaling Windows : erreurs de clic dues au zoom/display scaling (125%/150%), nécessitant une récupération fiable de la résolution réelle (ctypes).
- Équilibre : système trop strict (faux négatifs humains) vs trop laxiste (faux positifs bots) ; la **zone de doute** + second recours Gemini a stabilisé le compromis.

Malgré leurs performances impressionnantes sur des tâches complexes, les modèles de langage (LLM) se sont révélés parfois déroutants dans un contexte de développement : ils peuvent proposer des solutions élégantes sur le papier, tout en échouant à résoudre correctement des problèmes pourtant simples ou très concrets (gestion d'un cas limite, bug de logique, incohérence de format, donner des résultats sous la bonne forme, etc...).

À plusieurs reprises, il a donc fallu revenir au code, analyser l'exécution **à la main**, comprendre l'origine exacte du dysfonctionnement et corriger directement les scripts, y compris lorsque les suggestions de Gemini étaient incomplètes ou incorrectes.

Cette difficulté nous a appris une leçon importante : un LLM est un excellent accélérateur (idées, prototypes, aide à la syntaxe), mais il ne remplace pas la compréhension du code ni la phase de débogage rigoureuse, surtout dans un projet hybride (Frontend + API + ML + automatisation).

7 Bilan

7.1 Conclusion

Le projet **Captcha sous LLM** constitue une réussite technique : il combine une application web, une API, un modèle de machine learning et un bot attaquant, le tout orchestré par une interface d'administration.

Nous avons réussi à :

- Créer un dataset de mouvements (humains et bots).
- Entraîner un modèle capable de distinguer les deux avec une précision satisfaisante dans nos tests.
- Développer une application complète et ergonomique.

L'apport principal est la compréhension de la chaîne de la donnée : capture (React) → transformation (Python/Pandas) → apprentissage (Scikit-learn) → décision (API).

Ce travail nous a confrontés à des problématiques actuelles de cybersécurité et d'IA : un même ensemble d'outils peut servir à attaquer (bot réaliste) et à défendre (détection comportementale + modèle hybride).

Le projet nous a aussi appris à intégrer proprement des composants hétérogènes (web, ML, automatisation) dans une architecture cohérente.

7.2 Perspectives

- Renforcement du modèle : collecter plus de données et tester un modèle séquentiel (LSTM) pour mieux exploiter la dimension temporelle.
- Sécurité API : ajouter une authentification (token) et des contrôles anti-rejeu.
- Dockerisation : conteneuriser l'ensemble pour réduire les problèmes d'installation et de compatibilité.

8 Bibliographie

[1] Documentation interne du projet.

9 Webographie

[FAS] FastAPI Documentation, **Run a Server Manually / Deployment**, fastapi.tiangolo.com

[UVI] Uvicorn Documentation, uvicorn.dev

[SKL] Scikit-learn, **RandomForestClassifier** — **documentation**, scikit-learn.org

[GEM] Google AI for Developers, **Gemini API Docs**, ai.google.dev

[OPC] OpenCV Documentation, **OpenCV-Python Tutorials**, docs.opencv.org

[PAG] PyAutoGUI Documentation, **Quickstart / Cheat Sheet**, pyautogui.readthedocs.io

10 Annexes

Annexe A : Cahier des charges

A.1 Contexte et démarche initiale

Au démarrage du projet, les objectifs fonctionnels n'étaient pas encore clairement définis. Afin de structurer le travail, nous avons d'abord réalisé une maquette de l'application web (organisation des écrans, déroulement des étapes, comportement attendu du CAPTCHA) à l'aide de ChatGPT 5 et Claude 4.5 Sonnet.

Cette maquette nous a permis de valider l'expérience utilisateur et de clarifier le fonctionnement global avant de développer.

Une fois le concept stabilisé, nous avons refait le site de zéro dans l'objectif d'obtenir une interface plus **propre**, agréable et cohérente, tout en gardant les mêmes principes d'épreuves et de collecte des données à l'aide de Gemini 3.

A.2 Objectifs fonctionnels

Le site devait répondre aux besoins suivants :

- Proposer un CAPTCHA comportemental composé de plusieurs épreuves (Puzzle, Trace, Image-selector).
- Enregistrer les sessions utilisateurs avec l'ensemble des informations nécessaires (coordonnées, temps, etc.).
- Permettre à l'utilisateur, à la fin d'un CAPTCHA, de télécharger ses sessions individuellement (par jeu) et également une version cumulée (**stack**) regroupant toutes les sessions en un seul fichier (par jeu).
- Transmettre ces données à un serveur (backend) afin qu'un programme Python puisse analyser les sessions reçues et retourner une décision.

A.3 Données attendues (collecte & format)

Chaque épreuve génère une session contenant au minimum une série temporelle de points (par exemple : $x, y, timestamp$), permettant ensuite de calculer des indicateurs cinématiques (vitesse, accélération, hésitations, rectitude).

Les sessions doivent être exportables :

- Soit séparément (un fichier par épreuve).
- Soit regroupées (un fichier unique cumulant toutes les épreuves).

A.4 Constitution du jeu de données (phase de validation)

Pour valider rapidement la faisabilité, nous avons constitué un premier dataset minimal :

- 20 sessions humaines.
- 20 sessions générées par un bot LLM-VL.

Ce jeu initial avait pour objectif principal de vérifier que la chaîne **collecte** → **extraction** → **classification** fonctionne techniquement avant d'augmenter le volume de données.

A.5 Adversaire : Bot VL (vision + mouvements humains)

Le cahier des charges incluait volontairement un **attaquant** afin d'évaluer la robustesse du système. Ce bot devait :

- Prendre une capture d'écran.
- Utiliser OpenCV pour détecter l'épreuve/captcha à résoudre (repérage des zones et éléments de l'interface).
- Résoudre l'épreuve, puis demander à Gemini de générer/adapter une trajectoire de souris plus **humaine** (courbes, imperfections, micro-pauses), et exécuter ces mouvements OU demander à Gemini les réponses adéquates pour résoudre le captcha.

Plus précisément :

- Pour le **puzzle** : mettre le cube dans la bonne case, demander à Gemini de rendre les mouvements humains !
- Pour l'épreuve **image-selector** : utiliser Gemini et OpenCV pour identifier les bonnes images à sélectionner, puis cliquer en conservant un mouvement réaliste.
- Pour l'épreuve **Trace** : suivre une ligne en produisant des mouvements humains, non parfaitement rectilignes.

A.6 Backend et communication

Un backend devait être mis en place pour recevoir les fichiers de sessions envoyés par le client. L'objectif est que l'analyse s'effectue sur un ordinateur serveur exécutant un programme Python dédié.

A.7 Détection : modèle ML + arbitrage LLM (CAPTCHA intelligent)

La détection repose sur un modèle de Machine Learning entraîné en Python (Scikit-learn). L'organisation logicielle attendue est la suivante :

- Un script **utilitis** regroupant les commandes/fonctions clés (pré-traitement, extraction de features).
- Un script d'entraînement **train** produisant un modèle sérialisé (fichiers **.pkl**).
- Un script **api** chargé d'utiliser le modèle entraîné pour effectuer les prédictions.

Exposition de l'API via FastAPI et exécution via Uvicorn.

Fonctionnement de la décision :

1. L'API charge le modèle entraîné (**.pkl**).
2. Elle analyse les coordonnées reçues et renvoie une probabilité entre 0 et 1.
3. Plus la valeur est élevée, plus le comportement est considéré **bot** (et inversement, plus c'est faible, plus c'est **humain**).
4. Zone de doute : si la probabilité est entre 30% et 80%, le système déclenche un second avis via Gemini (LLM) ; c'est ce dernier qui tranche la décision finale.

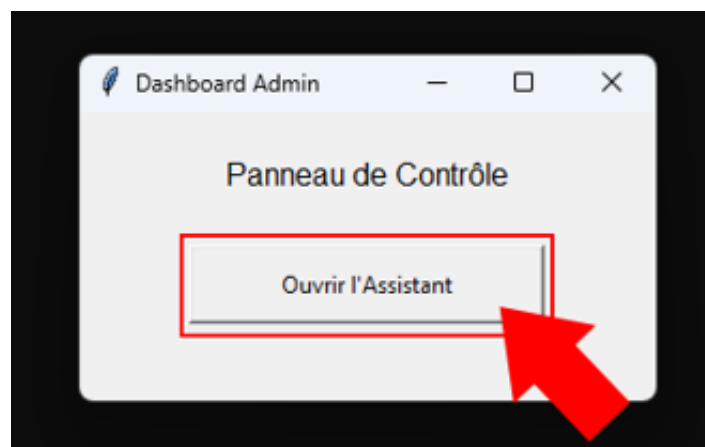
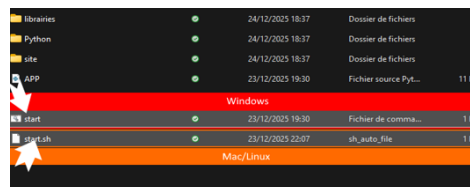
A.8 Hors périmètre (ce que le système ne doit pas faire)

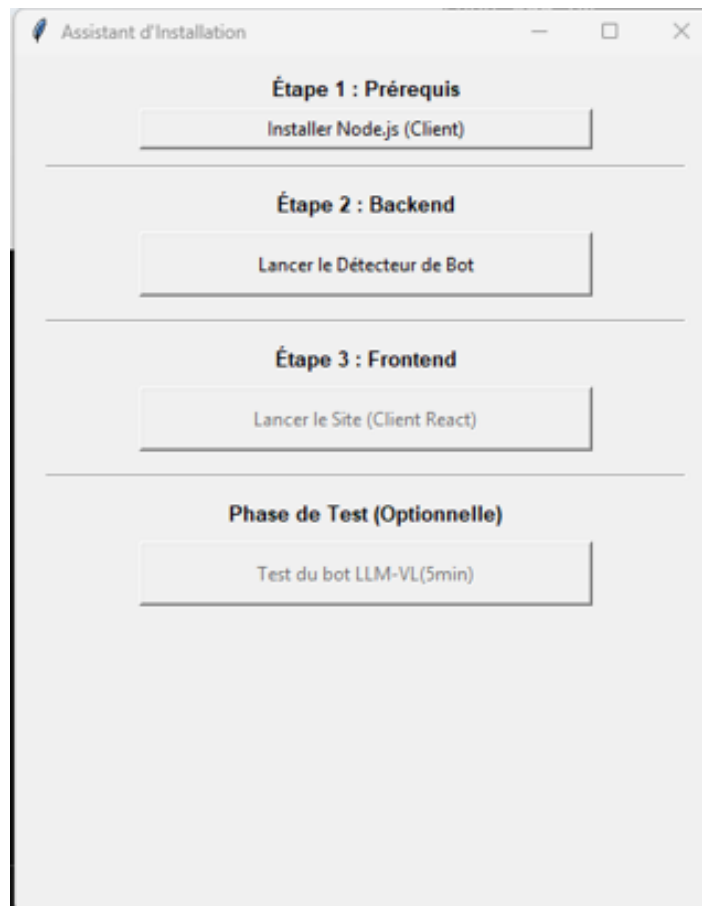
- Le système ne doit pas exiger de CAPTCHA **classiques** (texte déformé illisible, audio, etc.) : la détection doit rester principalement comportementale (mouvements de souris).

- Le système ne doit pas bloquer l'utilisateur sans explication : il doit toujours renvoyer un verdict explicite (**Humain** / **Robot**) accompagné d'un score/indice de confiance.
- Le backend ne doit pas dépendre uniquement du LLM : le modèle local (Random Forest) doit rester la première décision, et le LLM ne doit intervenir qu'en **zone de doute**.
- L'application ne doit pas nécessiter une configuration manuelle complexe pour être testée (objectif **lancement simple** via l'orchestrateur / scripts).
- Le système ne doit pas collecter des données personnelles directes (nom, email, etc.) : seules les données nécessaires à l'analyse de session (événements de souris et métriques) doivent être utilisées (RGPD).
- Le bot attaquant ne doit pas servir à des usages malveillants hors cadre du projet : il est uniquement un outil de test interne pour évaluer la robustesse du CAPTCHA.

Annexe B : Exemple d'exécution du projet

L'utilisateur lance l'application (Dashboard / lancement des services).





L'utilisateur ouvre l'interface web et démarre le CAPTCHA. L'utilisateur réalise les 3 épreuves : Puzzle, Sélection d'images, Trace.

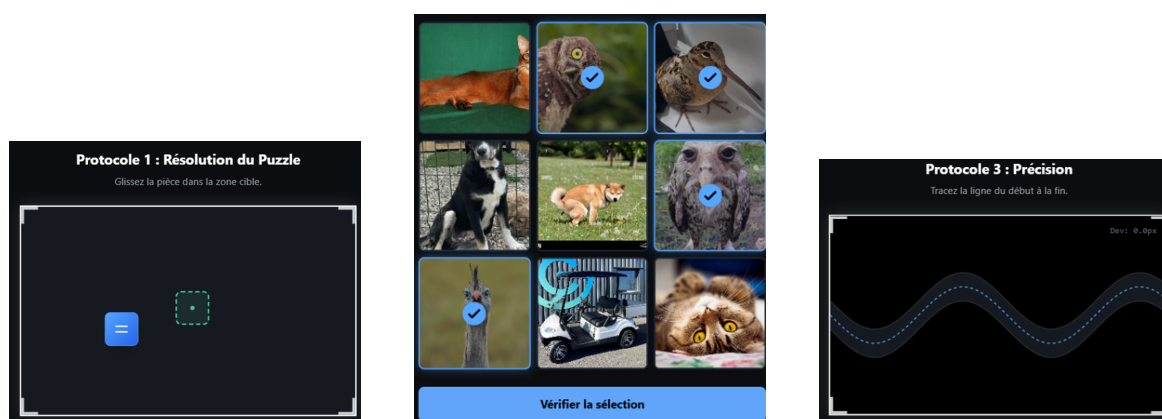


FIGURE 3 – Les épreuves

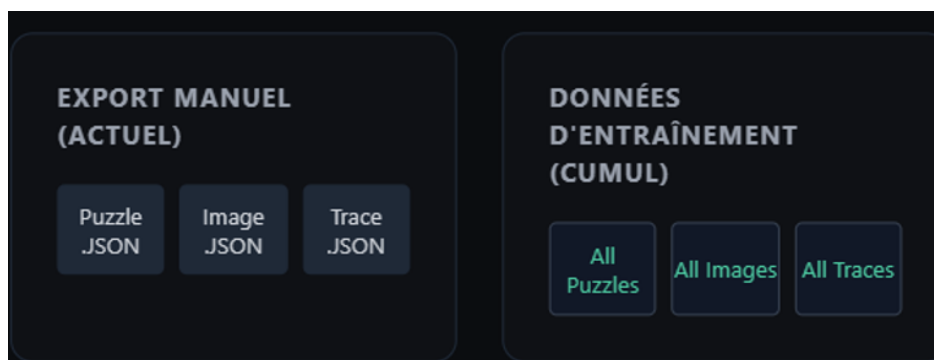
À la fin, une page **Résultats** s'affiche : elle résume la réussite des épreuves et affiche des indicateurs simples sur le mouvement envoyé à l'adresse `localhost:8000/analyze/${notre jeu}` qui va être reçu par notre `api.py`.

```

70 // Helper to send data (Using Secure Reverse lookup to get latest session)
71 const sendToPython = async (taskName: string, jsonData: any) => {
72   try {
73     console.log(`Sending payload for ${taskName}:`, JSON.stringify(jsonData).slice(
74
75     const response = await fetch(`http://localhost:8000/analyze/${taskName}`, {
76       method: 'POST',
77       headers: {
78         'Content-Type': 'application/json',
79       },
80       body: JSON.stringify(jsonData),
81     });
82
83     if (response.ok) {
84       const result = await response.json();
85       console.log(`API Result [${taskName}]`, result);
86       return result;
87     } else {
88       console.warn(`API Warning for ${taskName}:`, response.statusText);
89     }
90   }

```

L'utilisateur peut télécharger les fichiers de session (par épreuve) ainsi qu'un export d'historique pour l'entraînement/les tests.



L'utilisateur clique sur **Lancer la vérification** : les données sont envoyées au backend, puis un verdict final **Humain** / **Robot** est affiché avec un score de confiance.



En cas d'incertitude, le système effectue une vérification renforcée (CAPTCHA intelligent : second avis par LLM) avant de trancher.

```

# 4. Logique de Doute (Hybride)
if 0.30 < proba_bot < 0.80:
    print(f"🤖 Doute ({proba_bot:.2f}). Appel Gemini...")
    gemini_res = ask_gemini(payload, task_id)

    is_bot_gemini = gemini_res.get("is_bot", False)

    if is_bot_gemini:
        final_verdict = "ROBOT"
        # final_score = 0.95
        final_score = proba_bot + 0.2 # On pénalise un peu mais on garde la nuance
        if final_score > 1.0: final_score = 0.99
    else:
        final_verdict = "HUMAIN"
        # final_score = 0.1
        final_score = proba_bot - 0.2 # On bonifie un peu
        if final_score < 0.0: final_score = 0.01

else:

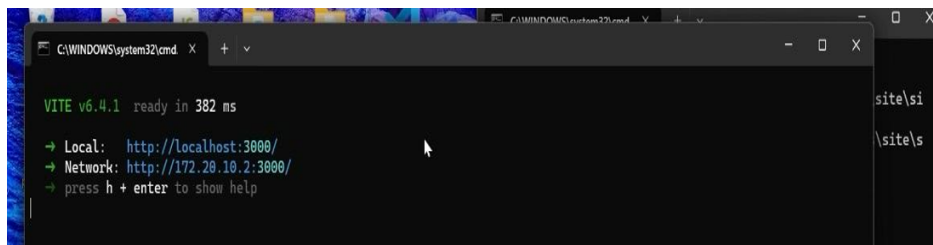
```

Annexe C : Manuel d'utilisation

Prérequis

- Le projet est compatible Windows (de préférence) / macOS / Linux (scripts `.bat` pour Windows et `.sh` pour macOS/Linux).
- Installer Python 3.10+ (Python 3.14 est à éviter).
- Installer Node.js (nécessaire au Frontend React) !
- Réseau : ports ouverts en local ET LIBRES ! (8000 pour l'API et 3000 pour le Frontend).
- Une clé API Gemini si l'analyse **second avis** est activée (en cas de **zone de doute**, la réponse peut être plus lente car un appel externe est effectué).

Avant de lancer un test, vérifier que le site est accessible à l'URL locale (ex : <http://localhost:3000>).

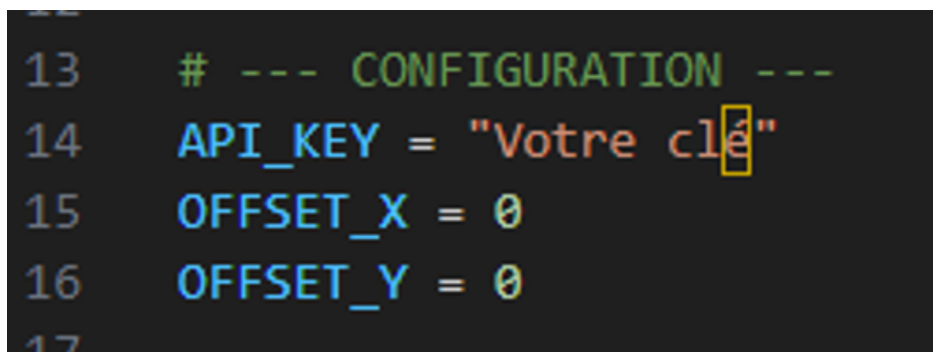


Installation (première exécution)

Se placer à la racine du projet. Installer toutes les dépendances :

- Backend : installation des dépendances Python (incluant FastAPI et Uvicorn, qui ont causé des erreurs lorsqu'ils n'étaient pas présents dans la liste des dépendances).
- Placer la clé Gemini dans les scripts suivants : `LLM-VL-1.py`, `LLM-VL-2.py` dans `/site/Python/Trainmodel`.

Lancer le test.sh	✓	27/12/2025 20:59	sh_auto_file	1 Ko
LLM-VL-1	✓	16/12/2025 13:53	Fichier source Pyt...	7 Ko
LLM-VL-2	✓	16/12/2025 13:53	Fichier source Pyt...	10 Ko
LLM-VL-3	✓	17/12/2025 16:20	Fichier source Pyt...	6 Ko



- Ainsi que `API.py` dans `/site/Python/Verifbot/src`.

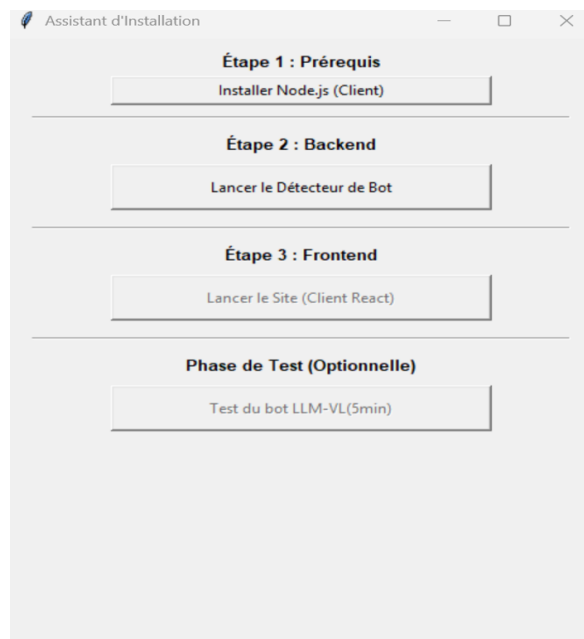
```

20 # --- TA CLÉ GEMINI ---
21 GOOGLE_API_KEY = "Votre clé"
22 genai.configure(api_key=GOOGLE_API_KEY)
23

```

Lancement via Dashboard (recommandé)

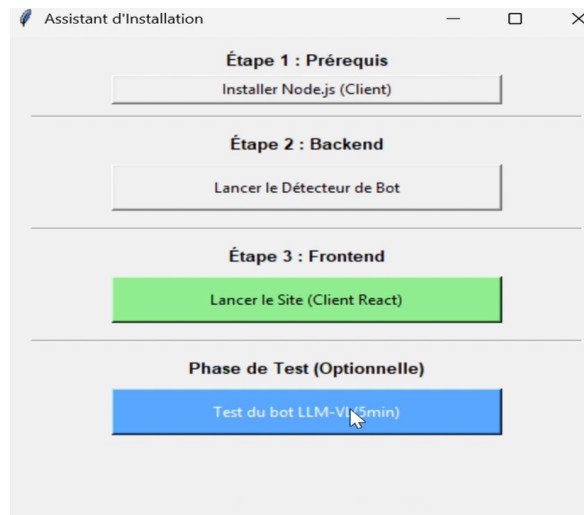
1. Lancer le Dashboard : exécuter `start.bat` (ou exécuter `start.sh` sous macOS/Linux).
2. Si le `start.bat` ou `start.sh` ne fonctionne pas, ouvrez `APP.py` (`python APP.py` dans le CMD).
3. Cliquer sur **Ouvrir l'Assistant**.
4. (Si nécessaire) cliquer sur **Installer Node.js (Client)**.
5. Cliquer sur **Lancer le Détecteur de Bot** (Backend API).
6. Cliquer sur **Lancer le Site (Client React)** (Frontend).



Votre site est maintenant accessible (ça peut prendre du temps en fonction de la puissance du PC) au lien suivant : <http://localhost:3000>.

Optionnel : test automatique du bot (LLM-VL)

Cliquer sur le bouton **Test du bot LLM-VL (5 min)** (bouton bleu).



Le système ouvre le navigateur, le passe en plein écran, clique sur **Initialiser**, puis exécute successivement `LLM-VL-1.py`, `LLM-VL-2.py`, `LLM-VL-3.py`. Ne touchez à rien et laissez la magie s'opérer.

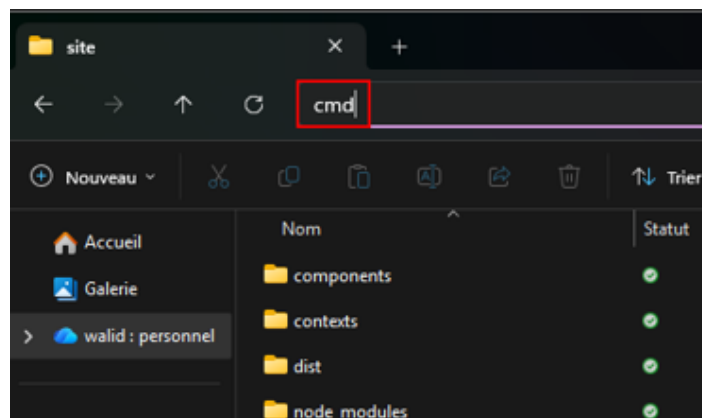
Comportement selon le système :

- Windows : exécution via scripts `.bat` (ouverture d'un terminal `cmd`).
- macOS / Linux : exécution via scripts `.sh`, avec tentative automatique de rendu exécutable (équivalent à `chmod +x`, requis sur macOS/Linux pour lancer un script).

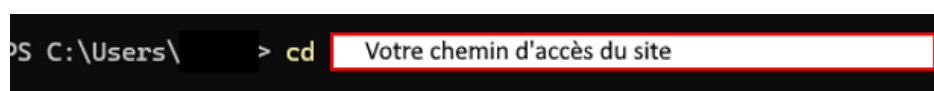
Lancement manuel (si besoin, sans Dashboard)

Cette méthode sert de plan B en cas de blocage du Dashboard.

Lancement du Site : Lancer le site : pour cela localisez l'emplacement du site (`site/site`), tapez `cmd` dans la barre d'adresse.



Ou lancer le CMD/Terminal et accéder au dossier où le site se situe en utilisant `cd`.



Puis exécutez les commandes suivantes :

- `npm install`
- `npm run build`

— `npm run dev`

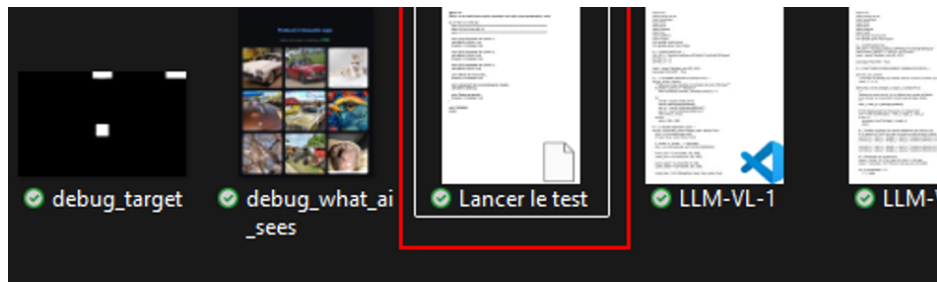
Votre site est maintenant accessible au lien suivant : <http://localhost:3000>.

Lancement de l'API détecteur de bot : Backend : lancer l'API, `/site/Python/Verifbot/src/Api` avec Uvicorn. Vérification : accéder ensuite à la documentation interactive FastAPI via `/docs` (ex. <http://localhost:8000/docs>).

Utilisation (passage du CAPTCHA)

1. Ouvrir le site (lien local) et cliquer sur le bouton d'initialisation/démarrage du test.
2. Réaliser les épreuves (tracé, puzzle, sélection d'images) de manière naturelle.
3. Le backend renvoie un verdict Humain ou Robot ; en cas de doute, une vérification supplémentaire via Gemini peut être déclenchée (temps de réponse plus long).

Optionnel : vous pouvez lancer un test par LLM en cliquant sur **Lancer le test.cmd** (ou **Lancer le test.sh** sur Linux/mac) dans `/site/Python/Trainmodel`.



Accédez au site et cliquez sur **Initialiser le protocole** et laissez la magie s'opérer. Ou ouvrez manuellement :

- `LLM-VL-1.py` pour le puzzle, donc le protocole 1.
- `LLM-VL-2.py` pour le protocole 2.
- `LLM-VL-3.py` pour le protocole 3.

Point important : mettre la fenêtre en plein écran et initialiser le protocole avant de lancer certains tests (sinon la détection/automatisation peut être perturbée et confondre par des icônes aux alentours).